

TUGAS PRAKTIKUM STRUKTUR DATA



Oleh:

Naufal Arkan Octyandi

2511533005

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU: DR. WAHYUDI, S.T, M.T

ASISTEN LABOR: SHERLY SUKMADIRA

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 8 JUNI 2026

Tugas BFS dan DFS

- Kode program java

```
1 package pekan9_2511533005;
2 import javax.swing.*;
3 import java.awt.*;
4 import java.util.*;
5 import java.util.List;
6
7 public class PetaKampus_2511533005 extends JFrame {
8     private final Map<String, List<String>> adjList_3005 = new HashMap<>();
9     private final Map<String, Point> positions_3005 = new HashMap<>();
10    private JComboBox<String> cbStart_3005;
11    private JComboBox<String> cbGoal_3005;
12    private JTextArea txtResult_3005;
13    private GraphPanel_3005 panelGraph_3005;
14    private List<String> visitedNodes_3005 = new ArrayList<>();
15    private List<String> pathNodes_3005 = new ArrayList<>();
16
17    public PetaKampus_2511533005() {
18        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS STEI ITB");
19        setSize(900, 750);
20        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21        setLocationRelativeTo(null);
22
23        JPanel mainPanel_3005 = new JPanel(new BorderLayout());
24        mainPanel_3005.setBackground(new Color(235, 240, 245));
25        setContentPane(mainPanel_3005);
26
27        initGraph_3005();
28
29        JPanel northContainer = new JPanel(new BorderLayout());
30        northContainer.setOpaque(false);
31
32        JPanel banner_3005 = new JPanel(new BorderLayout());
33        banner_3005.setBackground(new Color(27, 79, 114));
34        banner_3005.setPreferredSize(new Dimension(900, 45));
35
36        JLabel lblTitle_3005 = new JLabel("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS STEI ITB", JLabel.CENTER);
37        lblTitle_3005.setFont(new Font("SansSerif", Font.BOLD, 16));
38        lblTitle_3005.setForeground(Color.WHITE);
39        banner_3005.add(lblTitle_3005, BorderLayout.CENTER);
40
41        banner_3005.setPreferredSize(new Dimension(900, 45));
42
43        JLabel lblTitle_3005 = new JLabel("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS STEI ITB", JLabel.CENTER);
44        lblTitle_3005.setFont(new Font("SansSerif", Font.BOLD, 16));
45        lblTitle_3005.setForeground(Color.WHITE);
46        banner_3005.add(lblTitle_3005, BorderLayout.CENTER);
47
48        northContainer.add(banner_3005, BorderLayout.NORTH);
49
50        JPanel topPanel_3005 = new JPanel(new GridBagLayout());
51        topPanel_3005.setOpaque(false);
52        topPanel_3005.setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));
53        GridBagConstraints gbc_3005 = new GridBagConstraints();
54        gbc_3005.insets = new Insets(5, 5, 5, 5);
55        gbc_3005.fill = GridBagConstraints.HORIZONTAL;
56
57        JLabel lblStart_3005 = new JLabel("Lokasi Awal : [ ]");
58        lblStart_3005.setFont(new Font("SansSerif", Font.BOLD, 14));
59        gbc_3005.gridx = 0; gbc_3005.gridy = 0;
60        topPanel_3005.add(lblStart_3005, gbc_3005);
61
62        String[] nodesArray_3005 = adjList_3005.keySet().toArray(new String[0]);
63        Arrays.sort(nodesArray_3005);
64        cbStart_3005 = new JComboBox<>(nodesArray_3005);
65        cbStart_3005.setFont(new Font("Arial", Font.PLAIN, 14));
66        gbc_3005.gridx = 1; gbc_3005.gridy = 0;
67        topPanel_3005.add(cbStart_3005, gbc_3005);
68
69        JLabel lblClose1_3005 = new JLabel("[ ]");
70        lblClose1_3005.setFont(new Font("Arial", Font.BOLD, 14));
71        gbc_3005.gridx = 2; gbc_3005.gridy = 0;
72        topPanel_3005.add(lblClose1_3005, gbc_3005);
73
74        JLabel lblGoal_3005 = new JLabel("Lokasi Tujuan : [ ]");
75        lblGoal_3005.setFont(new Font("Arial", Font.BOLD, 14));
76        gbc_3005.gridx = 0; gbc_3005.gridy = 1;
77        topPanel_3005.add(lblGoal_3005, gbc_3005);
78
79        cbGoal_3005 = new JComboBox<>(nodesArray_3005);
80        cbGoal_3005.setFont(new Font("Arial", Font.PLAIN, 14));
```

```

73     cbGoal_3005.setFont(new Font("Arial", Font.PLAIN, 14));
74     gbc_3005.gridx = 1; gbc_3005.gridy = 1;
75     topPanel_3005.add(cbGoal_3005, gbc_3005);
76
77     JLabel lblClose2_3005 = new JLabel("]");
78     lblClose2_3005.setFont(new Font("Arial", Font.BOLD, 14));
79     gbc_3005.gridx = 2; gbc_3005.gridy = 1;
80     topPanel_3005.add(lblClose2_3005, gbc_3005);
81
82     JButton btnBfs_3005 = new JButton("[ BFS ]");
83     btnBfs_3005.setBackground(new Color(144, 238, 144));
84     btnBfs_3005.setFont(new Font("Arial", Font.BOLD, 14));
85     gbc_3005.gridx = 3; gbc_3005.gridy = 0; gbc_3005.gridheight = 2;
86     gbc_3005.ipadx = 20; gbc_3005.ipady = 10;
87     topPanel_3005.add(btnBfs_3005, gbc_3005);
88
89     JButton btnDfs_3005 = new JButton("[ DFS ]");
90     btnDfs_3005.setBackground(new Color(255, 203, 114));
91     btnDfs_3005.setFont(new Font("Arial", Font.BOLD, 14));
92     gbc_3005.gridx = 4; gbc_3005.gridy = 0;
93     topPanel_3005.add(btnDfs_3005, gbc_3005);
94
95     JButton btnReset_3005 = new JButton("[ RESET ]");
96     btnReset_3005.setBackground(new Color(240, 128, 128));
97     btnReset_3005.setFont(new Font("Arial", Font.BOLD, 14));
98     gbc_3005.gridx = 5; gbc_3005.gridy = 0;
99     topPanel_3005.add(btnReset_3005, gbc_3005);
100
101     northContainer.add(topPanel_3005, BorderLayout.CENTER);
102     mainPanel_3005.add(northContainer, BorderLayout.NORTH);
103
104     JPanel centerWrapper = new JPanel(new BorderLayout());
105     centerWrapper.setOpaque(false);
106     centerWrapper.setBorder(BorderFactory.createEmptyBorder(0, 15, 0, 15));
107
108     panelGraph_3005 = new GraphPanel_3005();
109     centerWrapper.add(panelGraph_3005, BorderLayout.CENTER);
110     mainPanel_3005.add(centerWrapper, BorderLayout.CENTER);
111
112     JPanel southPanel_3005 = new JPanel(new BorderLayout());

```

```

112     JPanel southPanel_3005 = new JPanel(new BorderLayout());
113     southPanel_3005.setOpaque(false);
114     southPanel_3005.setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));
115
116     txtResult_3005 = new JTextArea(4, 50);
117     txtResult_3005.setEditable(false);
118     txtResult_3005.setOpaque(false);
119     txtResult_3005.setFont(new Font("Monospaced", Font.BOLD, 16));
120     resetGraph_3005();
121
122     southPanel_3005.add(txtResult_3005, BorderLayout.CENTER);
123     mainPanel_3005.add(southPanel_3005, BorderLayout.SOUTH);
124
125     btnBfs_3005.addActionListener(e -> BFS_3005());
126     btnDfs_3005.addActionListener(e -> DFS_3005());
127     btnReset_3005.addActionListener(e -> resetGraph_3005());
128 }
129
130 private void initGraph_3005() {
131     positions_3005.put("LabDasar", new Point(220, 120));
132     positions_3005.put("STEILab", new Point(450, 120));
133     positions_3005.put("LabAI", new Point(680, 120));
134     positions_3005.put("LabSinyal", new Point(220, 230));
135     positions_3005.put("RuangDosen", new Point(450, 230));
136     positions_3005.put("LabJaringan", new Point(680, 230));
137     positions_3005.put("PerpusSTEI", new Point(220, 340));
138     positions_3005.put("TataUsaha", new Point(450, 340));
139     positions_3005.put("AulaTimur", new Point(680, 340));
140     positions_3005.put("LabRobotik", new Point(450, 440));
141     for (String node : positions_3005.keySet()) {
142         adjList_3005.put(node, new ArrayList<>());
143     }
144     addEdge_3005("LabDasar", "STEILab");
145     addEdge_3005("LabDasar", "LabSinyal");
146     addEdge_3005("STEILab", "LabAI");
147     addEdge_3005("STEILab", "RuangDosen");
148     addEdge_3005("LabAI", "LabJaringan");
149     addEdge_3005("LabSinyal", "RuangDosen");
150     addEdge_3005("LabSinyal", "PerpusSTEI");
151     addEdge_3005("RuangDosen", "LabJaringan");

```

```

160
161 private void addEdge_3005(String u, String v) {
162     adjList_3005.get(u).add(v);
163     adjList_3005.get(v).add(u);
164 }
165
166 public void BFS_3005() {
167     visitedNodes_3005.clear();
168     pathNodes_3005.clear();
169     String start_3005 = (String) cbStart_3005.getSelectedItemAt();
170     String goal_3005 = (String) cbGoal_3005.getSelectedItemAt();
171     Queue<String> queue_3005 = new LinkedList<>();
172     Map<String, String> parentMap_3005 = new HashMap<>();
173
174     queue_3005.add(start_3005);
175     visitedNodes_3005.add(start_3005);
176     boolean found_3005 = false;
177     while (!queue_3005.isEmpty()) {
178         String current_3005 = queue_3005.poll();
179         if (current_3005.equals(goal_3005)) {
180             found_3005 = true;
181             break;
182         }
183         List<String> neighbors_3005 = new ArrayList<>(adjList_3005.get(current_3005));
184         Collections.sort(neighbors_3005);
185         for (String neighbor_3005 : neighbors_3005) {
186             if (!visitedNodes_3005.contains(neighbor_3005)) {
187                 visitedNodes_3005.add(neighbor_3005);
188                 parentMap_3005.put(neighbor_3005, current_3005);
189                 queue_3005.add(neighbor_3005);
190             }
191         }
192     }
193     if (found_3005) {
194         displayPath_3005(parentMap_3005, start_3005, goal_3005, "BFS");
195     } else {
196         txtResult_3005.setText("Hasil Pencarian : BFS\nJalur : Tidak Ditemukan\nNode Dikunjungi : " + visitedNodes_3005 + "\nJumlah Node Dikunjungi : 0");
197     }
198     panelGraph_3005.repaint();
199 }

```

```

235 }
236
237 public void displayPath_3005(Map<String, String> parentMap, String start, String goal, String method) {
238     String curr_3005 = goal;
239     while (curr_3005 != null) {
240         pathNodes_3005.add(0, curr_3005);
241         curr_3005 = parentMap.get(curr_3005);
242     }
243     StringBuilder sb_3005 = new StringBuilder();
244     sb_3005.append("Hasil Pencarian : ").append(method).append("\n");
245     sb_3005.append("Jalur : ");
246     for (int i = 0; i < pathNodes_3005.size(); i++) {
247         sb_3005.append(pathNodes_3005.get(i));
248         if (i < pathNodes_3005.size() - 1) sb_3005.append(" -> ");
249     }
250     sb_3005.append("\nNode Dikunjungi : ");
251     for (int i = 0; i < visitedNodes_3005.size(); i++) {
252         sb_3005.append(visitedNodes_3005.get(i));
253         if (i < visitedNodes_3005.size() - 1) sb_3005.append(", ");
254     }
255     sb_3005.append("\nJumlah Node Dikunjungi : ").append(visitedNodes_3005.size());
256     txtResult_3005.setText(sb_3005.toString());
257 }
258
259 public void resetGraph_3005() {
260     visitedNodes_3005.clear();
261     pathNodes_3005.clear();
262     txtResult_3005.setText("Hasil Pencarian : \nJalur : \nNode Dikunjungi : \nJumlah Node Dikunjungi : 0");
263     if (panelGraph_3005 != null) panelGraph_3005.repaint();
264 }
265
266 private class GraphPanel_3005 extends JPanel {
267     public GraphPanel_3005() {
268         setBackground(color.WHITE);
269     }
270
271     @Override
272     protected void paintComponent(Graphics g) {
273         super.paintComponent(g);
274         Graphics2D g2d_3005 = (Graphics2D) g;

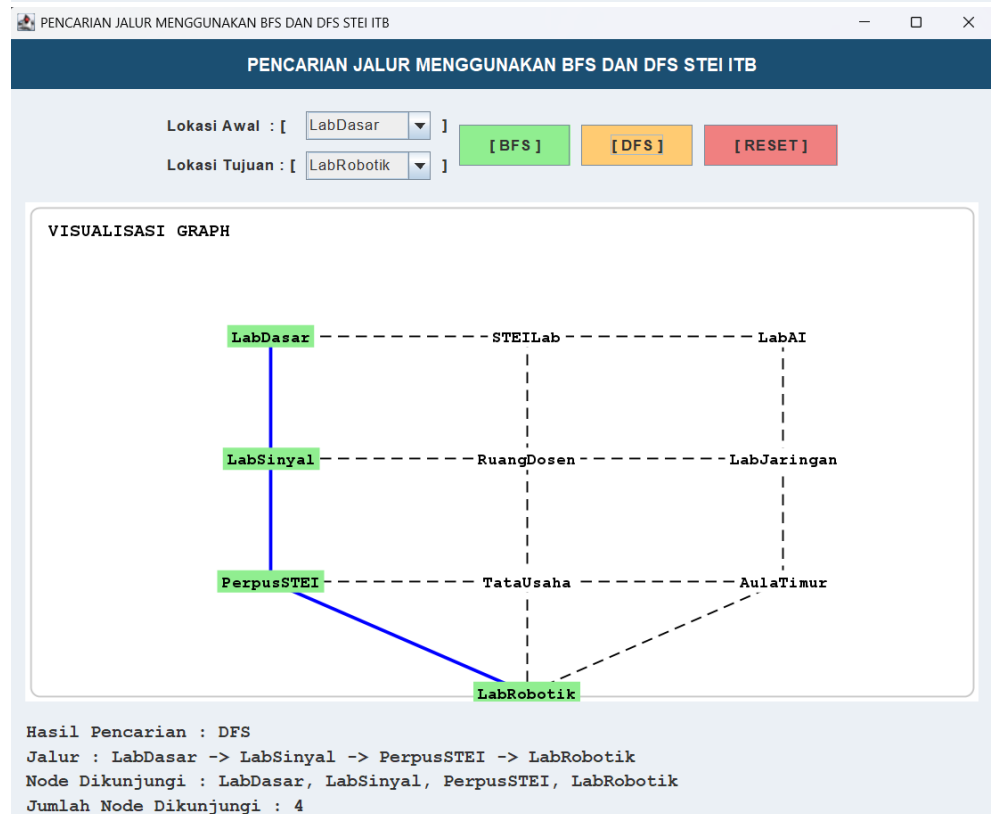
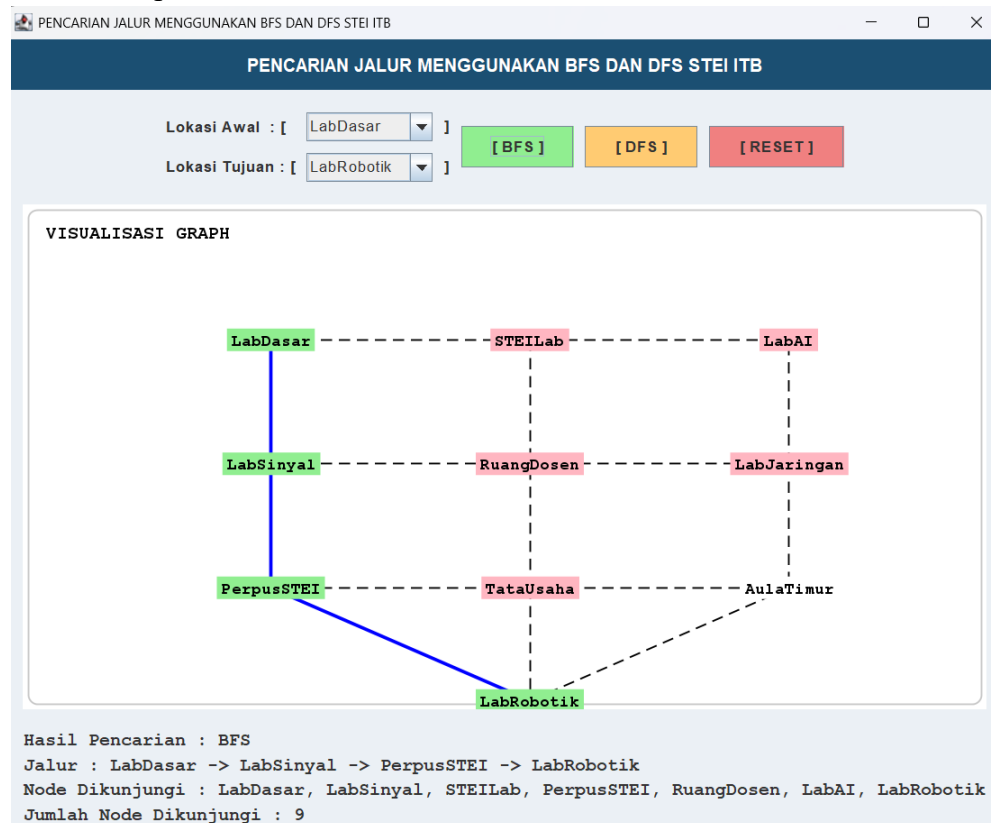
```

```

271         @Override
272         protected void paintComponent(Graphics g) {
273             super.paintComponent(g);
274             Graphics2D g2d_3005 = (Graphics2D) g;
275             g2d_3005.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
276
277             g2d_3005.setColor(new Color(200, 200, 200));
278             g2d_3005.setStroke(new BasicStroke(1.5f));
279             g2d_3005.drawRoundRect(5, 5, getWidth() - 10, getHeight() - 10, 15, 15);
280
281             g2d_3005.setColor(Color.BLACK);
282             g2d_3005.setFont(new Font("Monospaced", Font.BOLD, 16));
283             g2d_3005.drawString("VISUALISASI GRAPH", 20, 30);
284
285             Set<String> drawnEdges_3005 = new HashSet<>();
286             float[] dashPattern_3005 = {10.0f, 6.0f};
287             Stroke dashedStroke_3005 = new BasicStroke(1.5f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dashPattern_3005, 0.0f);
288             Stroke solidStroke_3005 = new BasicStroke(3.0f);
289
290             for (String u_3005 : adjList_3005.keySet()) {
291                 Point pU_3005 = positions_3005.get(u_3005);
292                 for (String v_3005 : adjList_3005.get(u_3005)) {
293                     String edgeKey_3005 = u_3005.compareTo(v_3005) < 0 ? u_3005 + "-" + v_3005 : v_3005 + "-" + u_3005;
294                     if (!drawnEdges_3005.contains(edgeKey_3005)) {
295                         Point pV_3005 = positions_3005.get(v_3005);
296                         boolean isEdgeInPath = false;
297                         if (pathNodes_3005.contains(u_3005) && pathNodes_3005.contains(v_3005)) {
298                             int idxU = pathNodes_3005.indexOf(u_3005);
299                             int idxV = pathNodes_3005.indexOf(v_3005);
300                             if (Math.abs(idxU - idxV) == 1) {
301                                 isEdgeInPath = true;
302                             }
303                         }
304
305                         if (isEdgeInPath) {
306                             g2d_3005.setColor(Color.BLUE);
307                             g2d_3005.setStroke(solidStroke_3005);
308                         } else {
309                             g2d_3005.setColor(Color.BLACK);
310                             g2d_3005.setStroke(dashedStroke_3005);
311
312                             g2d_3005.setStroke(dashedStroke_3005);
313                             g2d_3005.drawLine(pU_3005.x, pU_3005.y, pV_3005.x, pV_3005.y);
314                             drawnEdges_3005.add(edgeKey_3005);
315                         }
316                     }
317                 }
318
319                 g2d_3005.setFont(new Font("Monospaced", Font.BOLD, 14));
320                 FontMetrics fm_3005 = g2d_3005.getFontMetrics();
321                 for (Map.Entry<String, Point> entry_3005 : positions_3005.entrySet()) {
322                     String nodeName_3005 = entry_3005.getKey();
323                     Point p_3005 = entry_3005.getValue();
324                     int textWidth = fm_3005.stringWidth(nodeName_3005);
325                     int textHeight = fm_3005.getHeight();
326                     int textX = p_3005.x - (textWidth / 2);
327                     int textY = p_3005.y + (textHeight / 4);
328
329                     if (pathNodes_3005.contains(nodeName_3005)) {
330                         g2d_3005.setColor(new Color(144, 238, 144));
331                         g2d_3005.fillRect(textX - 4, p_3005.y - (textHeight/2), textWidth + 8, textHeight);
332                     } else if (visitedNodes_3005.contains(nodeName_3005)) {
333                         g2d_3005.setColor(new Color(255, 182, 193));
334                         g2d_3005.fillRect(textX - 4, p_3005.y - (textHeight/2), textWidth + 8, textHeight);
335                     } else {
336                         g2d_3005.setColor(Color.WHITE);
337                         g2d_3005.fillRect(textX - 4, p_3005.y - (textHeight/2), textWidth + 8, textHeight);
338                     }
339                     g2d_3005.setColor(Color.BLACK);
340                     g2d_3005.drawString(nodeName_3005, textX, textY);
341                 }
342             }
343
344             public static void main(String[] args) {
345                 SwingUtilities.invokeLater(() -> {
346                     new PetaKampus_2511533005().setVisible(true);
347                 });
348             }
349         }

```

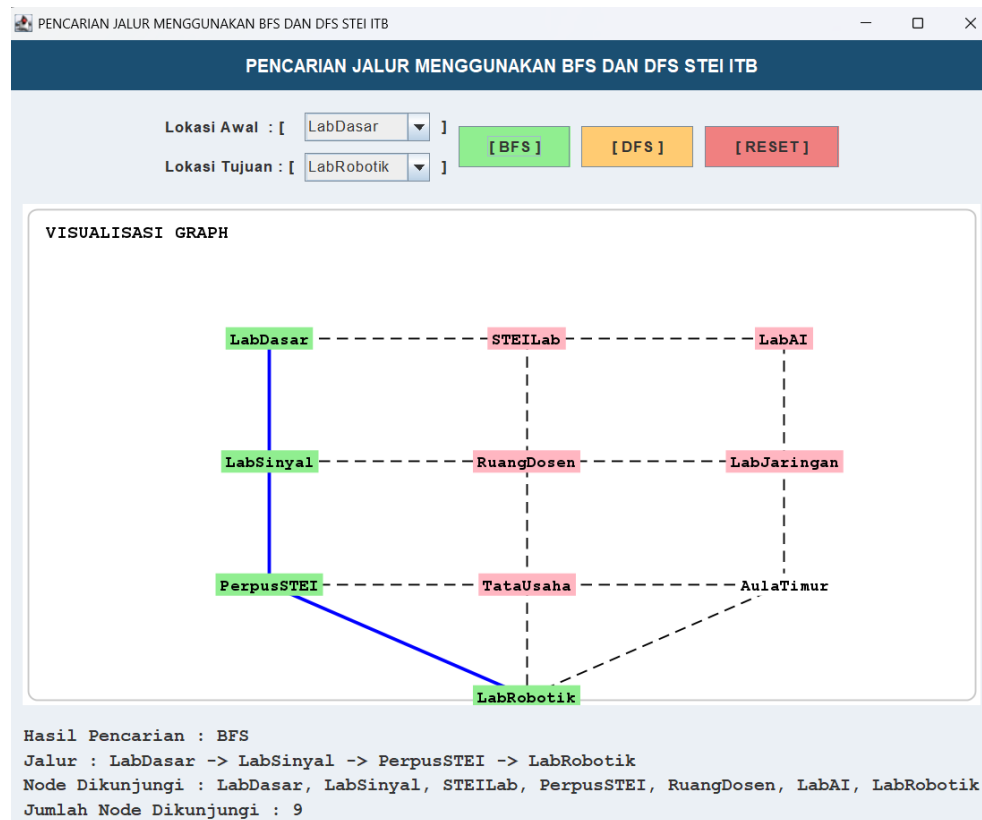
- Contoh output



- Deskripsi Peta
 Dalam tugas ini, peta lokasi yang saya ambil untuk dijadikan studi kasus nyata dari tugas praktikum ialah lingkungan Sekolah Teknik Elektro dan Informatika (STEIITB). Peta ini memodelkan letak gedung-gedung jurusan dan fasilitas utama yang sering dikunjungi mahasiswa. Model yang digunakan adalah undirected graph (graf tidak berarah), artinya jalan atau koridor yang menghubungkan satu gedung ke gedung lain bisa dilewati dua arah secara bebas (bolak-balik) tanpa ada batasan satu jalur.

- Jumlah Node (Simpul): 10 Node.
 Gedung yang dimasukkan meliputi LabDasar, STEILab, LabAI, LabRobotik, LabSinyal, RuangDosen, LabJaringan, PerpusSTEI, TataUsaha, AulaTimur.
- Jumlah Edge (Sisi): 15 Edge.
 Jalur penghubungnya meliputi
 LabDasar ke STEILab
 LabDasar ke LabSinyal
 STEILab ke LabAI
 STEILab ke RuangDosen
 LabAI ke LabJaringan
 LabSinyal ke RuangDosen
 LabSinyal ke PerpusSTEI
 RuangDosen ke LabJaringan
 RuangDosen ke TataUsaha
 LabJaringan ke AulaTimur
 PerpusSTEI ke TataUsaha
 TataUsaha ke AulaTimur
 PerpusSTEI ke LabRobotik
 TataUsaha ke LabRobotik
 AulaTimur ke LabRobotik

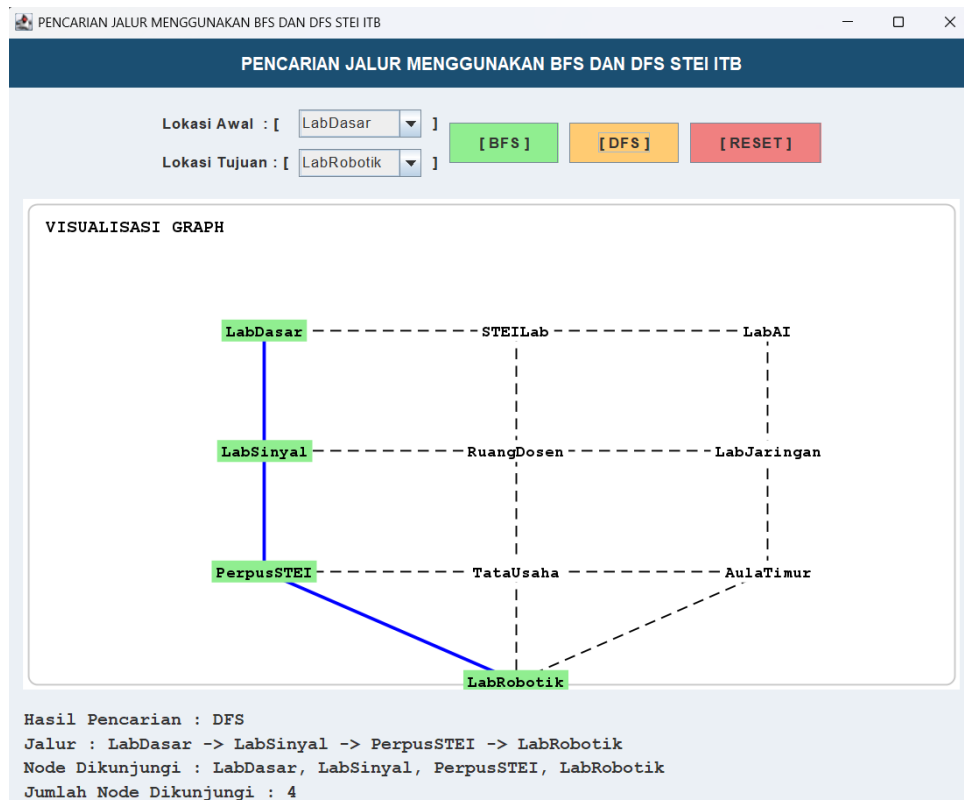
- Hasil BFS



Ketika pengujian dilakukan menggunakan metode Breadth-First Search dengan menetapkan titik awal dari PerpustakaanSTEI menuju titik tujuan LabAI, sistem berhasil menemukan jalur optimal yang terbentuk melalui rute PerpustakaanSTEI -> LabSinyal -> STEILab -> LabAI. Dalam proses pencarian ini, algoritma menyisir dan mengunjungi seluruh simpul yang ada di dalam graf dengan urutan yaitu PerpustakaanSTEI, LabSinyal, TataUsaha, LabRobotik, LabDasar, RuangDosen, AulaTimur, STEILab, LabJaringan, dan terakhir mengunci LabAI, sehingga total node yang dieksplorasi berjumlah sepuluh node.

Algoritma BFS beroperasi secara melebar menggunakan bantuan struktur data antrian (queue), di mana pencarian meronda seluruh node tetangga pada level yang sama secara horizontal terlebih dahulu sebelum melangkah lebih dalam, sehingga meskipun harus memeriksa seluruh simpul yang ada di peta, algoritma ini memberikan jaminan akurat bahwa jalur akhir yang terbentuk merupakan rute terpendek dengan jumlah lompatan terkecil (hanya melewati 3 sisi penghubung).

- Hasil DFS



Ketika pengujian dilakukan menggunakan metode Depth-First Search dengan menetapkan titik awal dari PerpustakaanSTEI menuju titik tujuan LabAI, sistem berhasil menemukan jalur yang terbentuk melalui rute PerpustakaanSTEI -> LabSinyal -> LabDasar -> STEILab -> LabAI. Dalam proses pencarian ini, algoritma menelusuri jalur secara mendalam ke satu arah terlebih dahulu dengan urutan kunjungan yaitu PerpustakaanSTEI, LabSinyal, LabDasar, STEILab, dan langsung mengunci LabAI begitu target ditemukan, sehingga total node yang dieksplorasi hanya berjumlah lima node. Algoritma DFS beroperasi secara mendalam menggunakan bantuan struktur data tumpukan (stack), di mana pencarian menelusuri simpul anak sedalam mungkin ke bawah sebelum melakukan proses mundur (backtracking), sehingga meskipun proses eksplorasinya jauh lebih hemat dalam jumlah node yang diperiksa pada kasus ini, algoritma ini tidak memberikan jaminan bahwa jalur akhir yang didapatkan merupakan rute terpendek, melainkan sekadar rute pertama yang berhasil menyentuh target tujuan.

- Kesimpulan Perbandingan BFS dan DFS

Dari hasil pengujian dan analisis yang dilakukan pada peta STEI ITB ini, kelihatan jelas kalau kedua algoritma punya cara kerja yang bertolak belakang dalam mencari jalur. Algoritma BFS terasa sangat diandalkan kalau kita ingin mencari rute yang paling dekat atau optimal. Karena sifatnya yang menyisir semua tetangga terdekat dulu secara merata, BFS sukses menemukan jalur terpendek (PerpustakaanSTEI -> LabSinyal -> STEILab -> LabAI), walaupun konsekuensinya program harus memeriksa hampir

seluruh node yang ada di peta (total sepuluh node) sehingga proses eksplorasinya jadi lebih banyak.

Sementara itu, algoritma DFS memang bisa menemukan lokasi tujuan dengan jumlah eksplorasi node yang jauh lebih sedikit pada kasus ini, yaitu cuma lima node saja karena dia langsung menusuk ke rute paling dalam secara vertikal. Tapi kelemahannya, kalau arah tusukan jalurnya tidak pas, jalur akhir yang dihasilkan DFS rentan memutar atau kurang efisien dibanding jalur terpendek yang sebenarnya, karena DFS hanya mencari rute pertama yang berhasil menyentuh target tanpa memedulikan optimasi jarak. Kesimpulannya, untuk pemodelan navigasi atau pencarian jalan di dunia nyata seperti peta kampus ini, algoritma BFS jauh lebih cocok dan ideal digunakan ketimbang DFS karena ada jaminan akurat kalau rute yang diberikan kepada pengguna adalah jalur yang paling cepat dengan jumlah lompatan terkecil.